LAB 7 – FUZZ TESTING

NAME : PRUTHVIRAJ AJAY PATIL

SRN : PES1UG24CS345

SECTION : 5F

Objective

To use the Hypothesis library for fuzz testing, identify input-handling bugs in the irrigation scheduler, and verify the fixes with automated tests.

Tools Used

- Python 3, pytest and Hypothesis
- Visual Studio Code

Application Functions

- parse_zone() – reads a zone name and plot area.
- moisture_average() – calculates the average of moisture readings.
- water_needed() – calculates water required after rainfall.

Original Application Run

The program was first tested with valid inputs: north:120, 40,35,50 and 120|8.

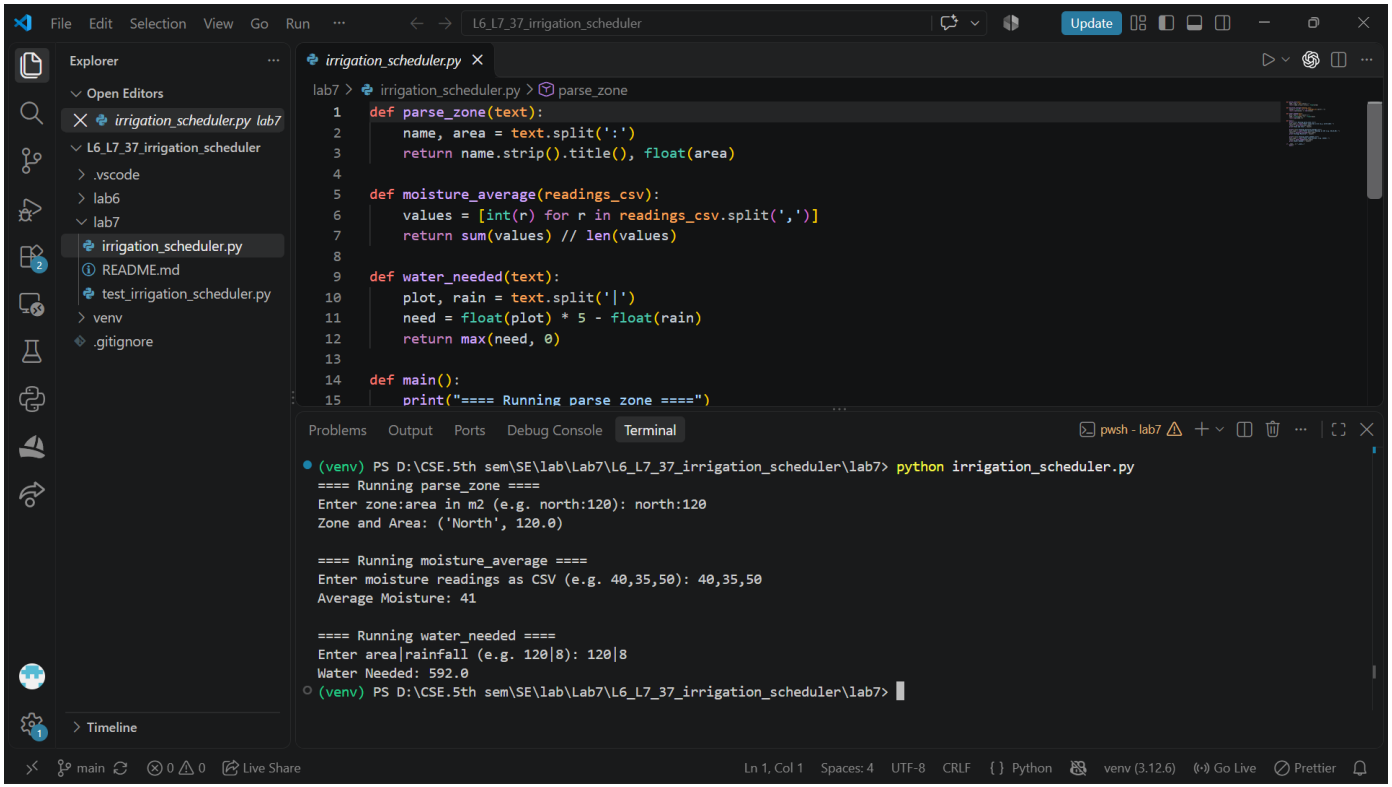


Figure 1. Original application output.

Fuzz Testing – Initial Run

Hypothesis generated different inputs for the three functions. The initial test run exposed problems such as handling None values, negative area and negative moisture readings.

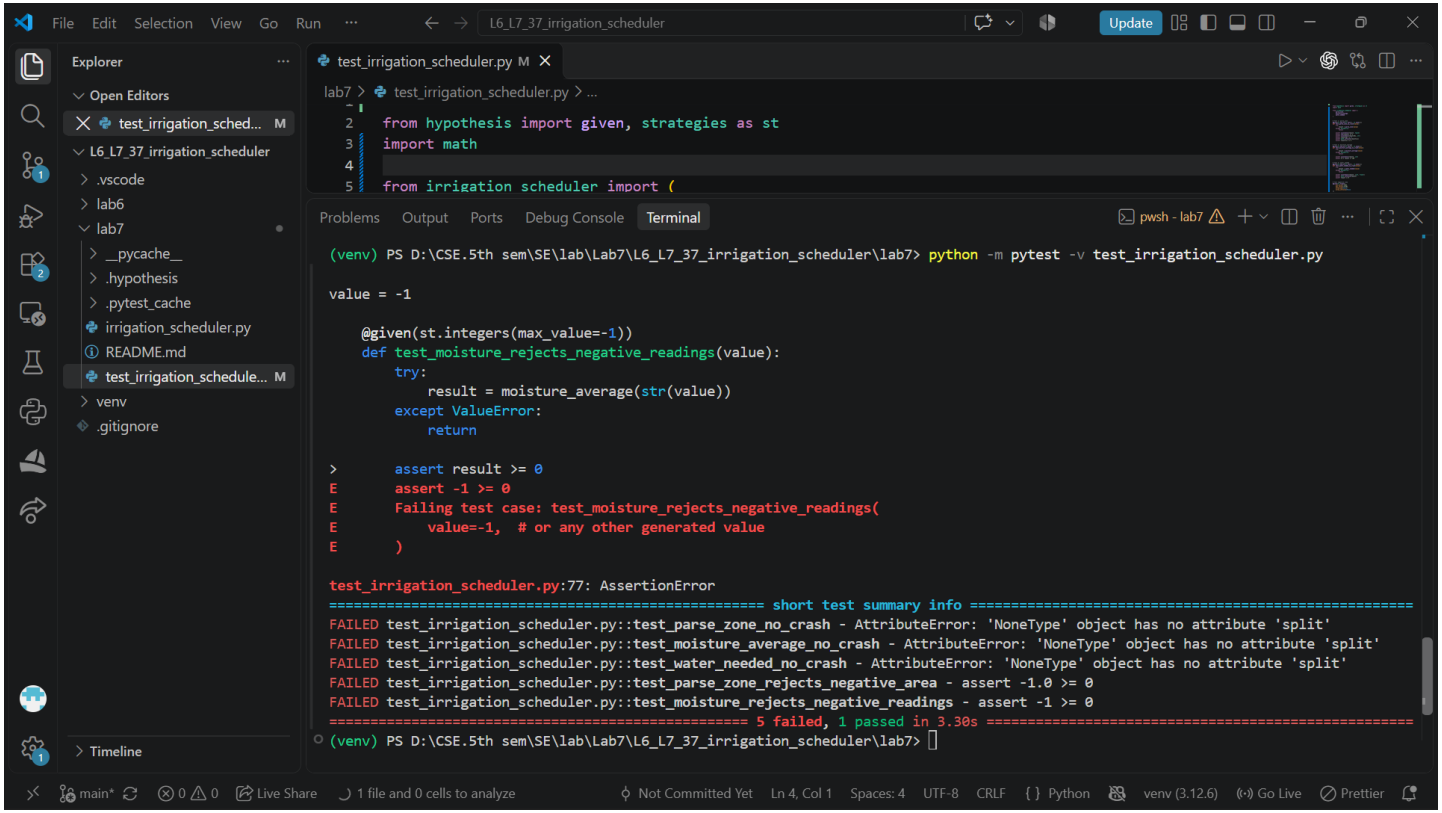


Figure 2. Initial pytest output: five failures and one passing test.

Issues Found

- Non-string inputs caused errors during string operations.
- Negative values were accepted for area and moisture readings.
- Input format and numeric values needed validation.

Application Fixes

Input validation was added to check data types, separators, empty fields, numeric conversion and valid ranges. Non-finite values and negative area or rainfall are rejected. Moisture readings are restricted to the range 0–100.

Updated Application Run

The application was run again with the same valid sample inputs to confirm that its normal calculations still worked.

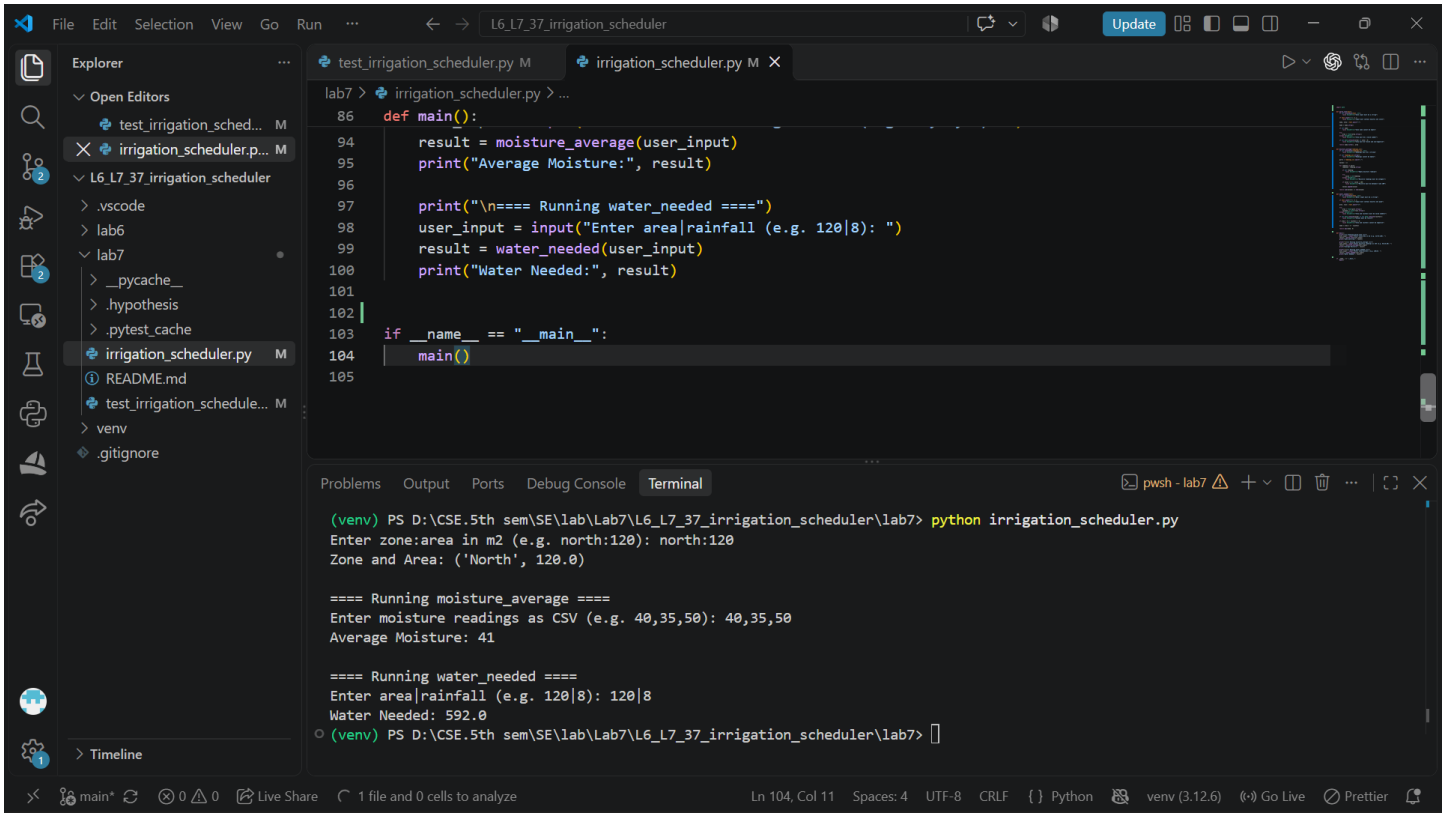


Figure 3. Updated application running with valid inputs.

Final Test Run

The updated application was tested again using pytest and Hypothesis. All six test functions passed.

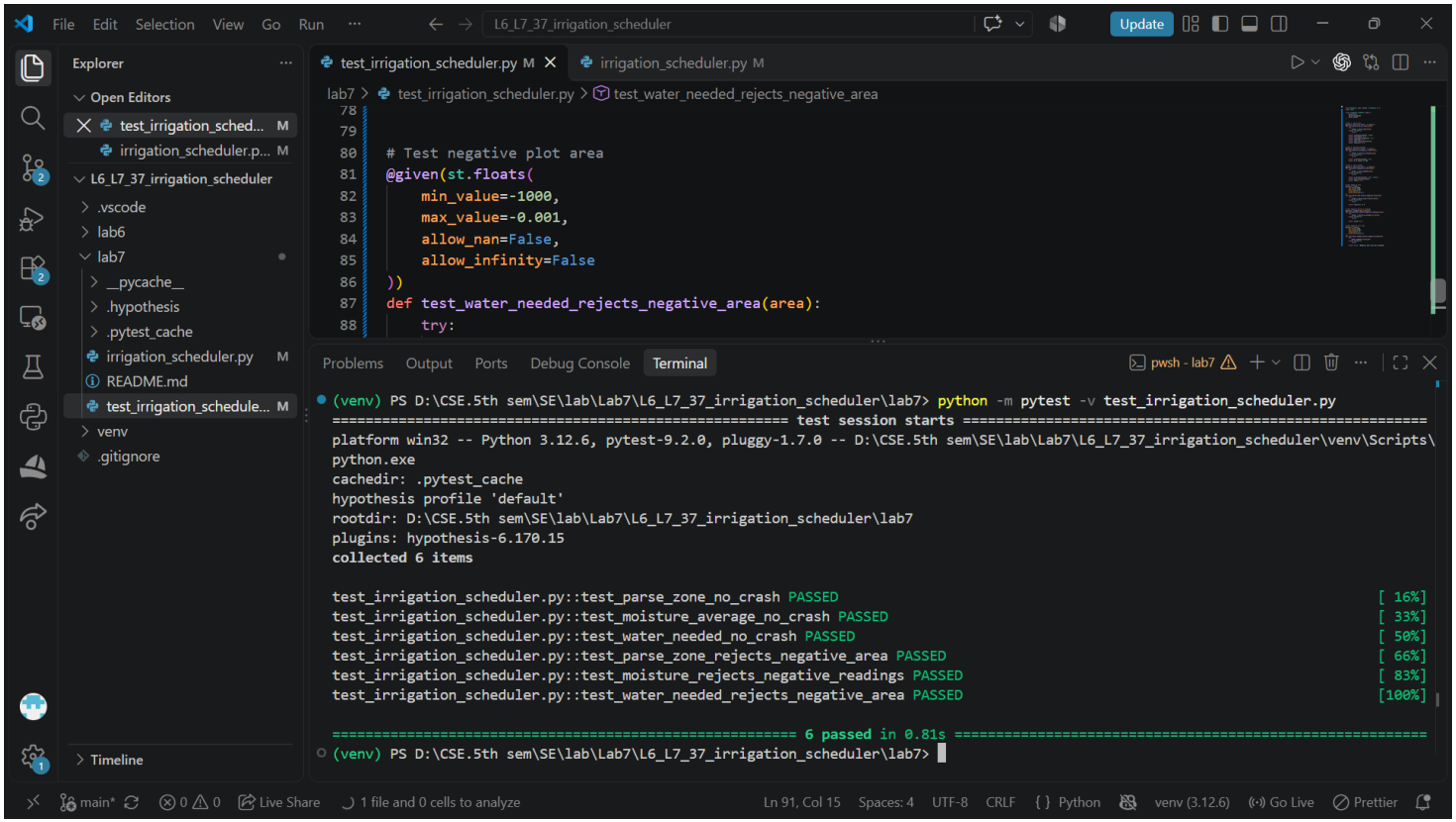


Figure 4. Final test result: six passed.

Result and Conclusion

Fuzz testing identified input-validation problems in the irrigation scheduler. After adding validation checks, all six test functions passed. This experiment showed how automatically generated inputs can help find bugs that may be missed by normal testing.